
SESSION-1

Task 2: To Analyse process abstraction, execute fork(), understand exec() family, analyse parent-child relationships, inspect process tree, practice system call tracing.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        printf("Executing ls using exec()\n");
        execlp("ls", "ls", "-l", NULL);
        perror("exec failed");
        exit(1);
    } else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");
        wait(NULL);
        printf("Child process completed.\n");
    }

    return 0;
}
```

OUTPUT:

```
sujit@sujit-LOQ-15IRX9: ~  
  
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano main4.c  
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ gcc main4.c -o main4  
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ ./main4  
Parent Process Started  
Parent PID: 32841  
  
Parent Process  
Parent PID: 32841  
Child PID: 32842  
Parent waiting for child...  
  
Child Process  
Child PID: 32842  
Parent PID: 32841  
Executing ls using exec()  
total 2220  
-rw-rw-r-- 1 sujit sujit 717983 Aug 20 14:21 'OSSP SWeek-4.docx.pdf'  
-rw-rw-r-- 1 sujit sujit 745432 Aug 20 14:53 'OSSP Skill Week2 .docx.pdf'  
-rwxrwxr-x 1 sujit sujit 16288 Aug 20 15:27 main4  
-rw-rw-r-- 1 sujit sujit 870 Aug 20 15:27 main4.c  
-rw-rw-r-- 1 sujit sujit 783873 Aug 20 15:14 'ossP SWeek-3.docx.pdf'  
Child process completed.  
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$
```

SESSION-2

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

A) Create a file by using

```
sujit@sujit-LOQ-15IRX9: ~  
  
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano main5.c  
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$
```

Then press enter and the code terminal will open then write a code

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    while (1) {
        printf("myShell> ");
        fgets(input, sizeof(input), stdin);

        // Remove newline
        input[strcspn(input, "\n")] = 0;

        // Exit condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting Shell...\n");
            break;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

After writing the code save the your program by ctrl O and for coming back to command press ctrl X then compile it by the command

```
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano main5.c
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano main5.c
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ gcc main5.c -o main5
```

After compiling to get the output enter command

```
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano main5.c
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano main5.c
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ gcc main5.c -o main5
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ ./main5
myShell> OSSP
You entered: OSSP
myShell> POWERSHELL
You entered: POWERSHELL
myShell> UBUNTU
You entered: UBUNTU
myShell> 
```

Task 2: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

CODE:

```

GNU nano 8.7.1
#include <stdio.h>

int main() {
    char buffer[100];
    int ch;
    int index = 0;

    printf("Type a command and press Enter:\n");
    printf("input> ");

    while ((ch = getchar()) != '\n' && ch != EOF) {
        if (ch == 127 || ch == 8) {
            if (index > 0) {
                index--;
            }
        } else if (index < 99) {
            buffer[index++] = ch;
        }
    }

    buffer[index] = '\0';

    printf("\nCommand entered: %s\n", buffer);
    printf("Characters stored: %d\n", index);

    return 0;
}

```

OUTPUT-:

```

sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano main7.c
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ gcc main7.c -o main7
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ ./main7
Type a command and press Enter:
input> Hello World

Command entered: Hello World
Characters stored: 11
sujit@sujit-L0Q-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ 

```